

Microprocessor

1) Diff. between microprocessor & microcontroller, & what is Von-Neumann architecture? 8086 & 8088 microprocessor.

Microprocessor

i) Microprocessor contains RAM, ROM, I/O ports on the chip itself.

ii) The addition of external RAM, ROM & I/O ports makes the whole system bulkier & much more expensive.

iii) It can't use in a small circuitry system. Microprocessor has the adv. of versatility, such that the designer can decide the amount of RAM, ROM & I/O ports as needed.

iv) Microprocessor has the adv. of versatility such that the designer can decide the amount of RAM, ROM & I/O ports as needed.

Microcontroller

• Microcontroller has a C.P.U. in addition to a fixed amount of RAM, ROM, I/O ports and timer all on a single chip.

• Because of embedded system there is no external RAM, ROM, timer, I/O ports & external memory in a microcontroller system. So it is less expensive.

• Microcontroller is ideal for much application in which cost & space are critical.

It is not possible in a microcontroller system.

Von-Neumann architecture design that forms the basis for most computer systems today.

i) CPU:- It is responsible for executing instructions. It fetches both data & instructions from memory.

ii) Memory:- The computer memory stores both data & instructions.

iii) Control unit:- It manages the flow of data & instructions between the CPU, memory & other peripherals.

iv) Arithmetic & Logic Unit (ALU):- It performs arithmetic & logical operation on data.

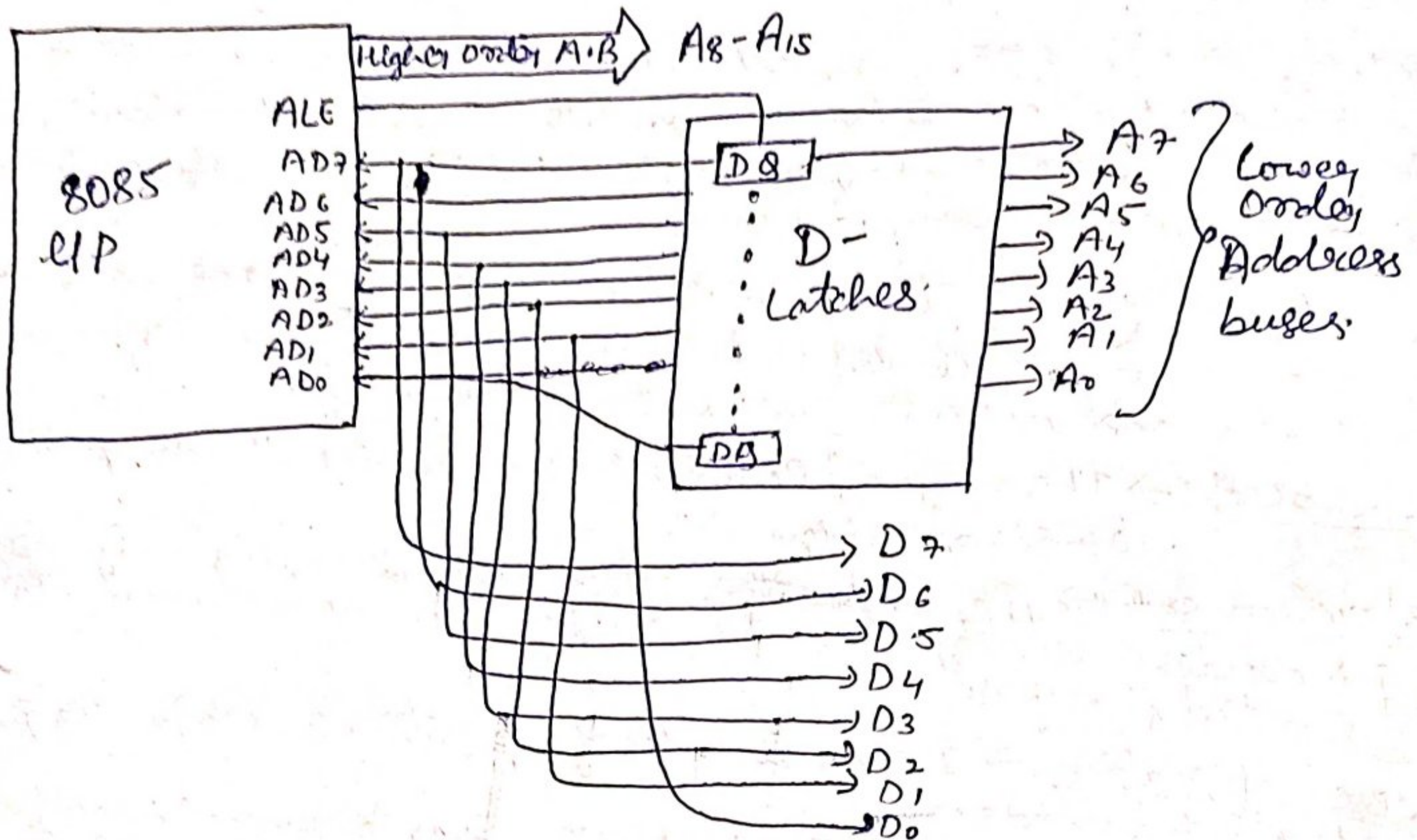
v) I/O:- Device such as keyboard, monitors & storage are connected to the comp. for input & output operations.

8086:-> The 8086 microprocessor is a 16-bit microprocessor that was introduced by intel in 1978. 2

- Architecture:- i) The 8086 has a Complex instruction set computing (CISC) architecture which means it supports a large set of instructions.
ii) It is a 16-bit processor, which means it can process data & instructions in 16-bit chunks at a time.
- Registers:-> i) The 8086 has a set of 16, 16-bit registers including general-purpose registers, segment registers & index registers.
ii) The general-purpose registers are AX, BX, CX & DX, each of which can be used for various operations.
- Memory Addressing:-> The 8086 uses a segmented memory model where the memory is divided into segments of 64 kilobytes.
ii) It can address up to 1 megabyte of memory.
- clock speed:-> The original 8086 processor operated at clock speeds of 5 or 8 MHz.
- Instruction set:-> The instruction set of the 8086 is relatively large & includes a variety of instructions for data manipulation, A.O. Control flow & more.
- Mode of operation:-> It has 2 modes of operation:- real mode & protected mode.
- Instruction pipelining:-> The 8086 uses a simple form of instruction pipelining, allowing it to overlap the execution of multiple instructions to improve performance.
- Peripheral support:-> It includes instructions & support for interfacing the various peripherals, making it suitable for a wide range of applications.
- ~~Legacy~~ 8088:- Architecture, Registers, Memory Addressing Same as 8086.
 - clock speed:-> 5, 8, 10 MHz.
 - Peripheral Support:-> The 8088 includes instructions & support for interfacing with various peripherals similar to the 8086.
 - Usage in IBM PC:-> The 8088 became widely known & used as the CPU.
 - Data Bus width:-> The 8088's external data bus is 8 bits wide which means it fetches or stores data in 8-bit increments even though it processes data internally as 16-bit.

2) Explain the demultiplexing the address/data multiplexed bus with the help of proper diagrams. Explain the function of ALE Pin of 8085 μ IP.

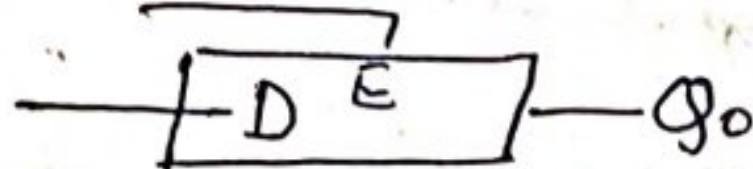
=> Multiplexed address & data bus ($A_{D0}-A_{D7}$)



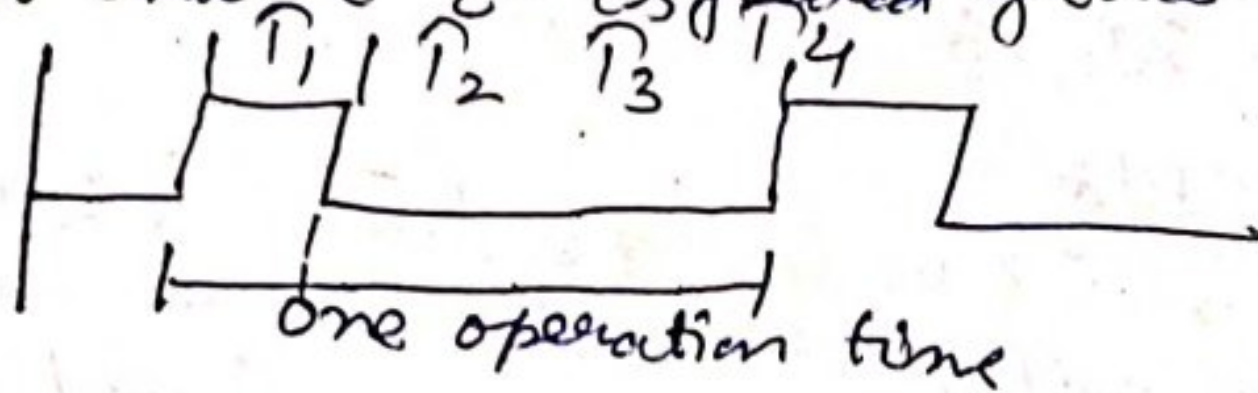
* Generally address bus (unidirectional) but after every operation acts as data bus as per clock (ALE).

* ALE :- (+)ve going pulse when μ IP is operating. It distinguishes the address bus & data address line Enable signal.

* Latch :-
(D-latch)



when enable 'E' is fed gives output



Working :-

- During T_1 , ALE is (+)ve & O/P is given address as given to A_0-A_{15} .
- After T_1 , ALE is 0, latch is disabled but address bus HOLDS the address given before until next (+)ve ALE is given i.e. another operation starts.
- After T_1 , i.e. ALE = 0 the lines are used as data buses.

$I/O/\bar{M}$:- (Input/output Memory) :- Differentiates if address on address bus is for memory or input/output.

when $I/O/\bar{M} = 0 \rightarrow$ Memory is accessed
 $I/O/\bar{M} = 1 \rightarrow I/O$ is accessed.

3

$S_0, S_1 \rightarrow$	S_1	S_0	operation
	0	0	Half operation
	0	1	write operation
	1	0	Read operation
	1	1	Fetch operation. (Read the opcode from memory)

$\overline{WR}$ (write) $\times$ $\overline{RD}$ (Read) :- Active mode signal
 write :- Write information in memory.
 Read :- Reads only external value.

3) Describe the different addressing modes of 8085, 8086 μ processor & 8051 microcontroller

Pdf $\Rightarrow$ Pg = 14

4) Draw & explain the flag structure of 8085 & 8086 μ processor & 8051 μ controller

2) Immediate addressing mode :- Here one of the operand is immediately specified in the instruction. $MOV A, MOV R_x$

data shift $\Rightarrow$ 00-FF

address of data memory & SFR 8bit = 00-7F
 80-FF

• Registered addressing mode :- When the operand of an instruction are registers only. $MOV A, R_0$
 $ADD A, R_0$

• Direct addressing mode :- When the address of the operand is directly specified in the instruction. $MOV A, 55H$
 $ADD A, 45H$

• Indirect addressing mode :- Here the address of the operand is not directly given but is placed in the register. $MOV A, @R_0$

• Index address mode :- To read the code from ROM this address is used. $MOV C, A, @ A + DPTR$
 For code

4) Draw & explain the flag structure of 8085 & 8086 μ processor & 8051 μ controller.

$\Rightarrow$

D_{15}	D_{14}	D_{13}	D_{12}	D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1
X	X	X	X	OF	DF	IF	TF	SF	ZF	X	AF	X	PF	CF

Control flag

• OF (Overflow flag)
 1 = Overflow occurred
 0 = No Overflow

• DF (Direction flag)
 1 = Auto Decrement
 0 = Auto Increment

Pop flag

• Interrupt flag (IF) $\Rightarrow$ 1 = Enable Interrupt
0 = Disable Interrupt

• SF (Sign Flag) $\Rightarrow$ 1 = MSB of result is 1 [-ve no]
0 = MSB of result is 0.

Used with signed operation.

• ZF (Zero Flag) $\Rightarrow$ 1 = Result is zero.
0 = Result is not zero.

• Auxiliary Carry flag $\Rightarrow$ 1 = Carry from lower to higher nibble
0 = No Auxiliary Carry
Applicable in 8 bits instructions

• Parity Flag $\Rightarrow$ 1 = Even parity
0 = odd parity

• Carry flag $\Rightarrow$ 0 = No carry
1 = Carry from MSB.

#

8085 I/P

D7	D6	D5	D4	D3	D2	D1	D0
S	Z		AC		P		CY

Flag is an 8-bit register containing 5-1 bit flags.

- Sign:- Set if the most significant bit of the result is set.
- Zero:- Set if the result is zero.
- Auxiliary Carry $\rightarrow$ Set if there was a carry out from bit 3 to bit 4 of the result.
- Parity $\rightarrow$ Set if the Parity (the no. of set bits in the result) is even.
- Carry:- Set if there was a carry during addition, or borrow during subtraction/comparison.

8085 I/P Controller

(MSB)	PSW.7	PSW.6	PSW.5	PSW.4	PSW.3	PSW.2	PSW.1	PSW.0	(LSB)
Direct Addressing	CY	AC	FO	RS1	RS0	OV	-	P	
Bit Address	D7	D6	D5	D4	D3	D2	D1	D0	
Carry flag									Parity Flag
Auxiliary carry flag									User Definable flag
General Purpose status flag									Overflow flag
									Register Bank Select Bit 1
									Register Bank Select Bit 0

Q5) Write a program to generate a 1msec, 1sec & 10msec time delay
consider the operating frequency of 8085 is 2MHz & 3MHz

$$\Rightarrow \text{Time delay (in s)} = \frac{\text{NO. of instructions} \times \text{Machine cycle time}}{\text{microprocessor frequency}}$$

8085 processor has an inst cycle of 3-6 machine cycle

Code $\rightarrow$ 4 zero
1 msec
ORG 0000H ; start address (8000)

MSEC_LOOP:

MVI B, 0FAH ; Load B register with 250 (0FAH)

MSEC_DELAY:

MVI C, 0FFH ; Load C register with 255 (0FFH)

MSEC_DELAY_LOOP:

NOP ; NO operation (1 machine cycle)

NOP ; NO operation (1 machine cycle)

NOP ; " " " " " "

DCR C ; Decrement C (" " ")

JNZ MSEC_DELAY_LOOP ; Jump if not zero (3 machine cycles)

DCR B ; Decrement B (1 machine cycle)

JNZ MSEC_DELAY ; Jump if not zero (3 m. cycles)

JMP SEC_LOOP ; Jump to 1-second delay.

2. 1 sec

SEC_LOOP:

MVI B, 0F4H ; Load B register with ²⁴⁴0F4H (0F4H)

SEC_DELAY:

MVI C, 0FFH ; Load C register with 255 (0FFH)

SEC_DELAY_LOOP:

NOP ; NO operation (1 machine cycle)

NOP ; " " " " " "

NOP ; " " " " " "

DCR C ; Decrement C (" " ")

JNZ SEC_DELAY_LOOP ; Jump if not zero (3 m. cycles)

~~START~~

DCRB ; Decrement B (1 machine cycle)
 JNZ SEC-DELAY ; Jump if not zero (3 machine cycles)
 JMP MSEC-10-LOOP ; Jump to 10-msec delay

3. For 10 msec

MSEC-10-LOOP;
 MUIB 00AH ; Load B register with 10 (00AH)
 MSEC-10-DELAY;
 MUI C : 0FFH ; Load C register with 255 (0FFH)
 MSEC-10-DELAY-LOOP;
 NOP ; NO operation (1 machine cycle)
 NOP ; " " " " " "
 NOP ; " " " " " "
 DCR C ; Decrement C (1 machine cycle)
 JNZ MSEC-10-DELAY-LOOP ; Jump if not zero (3 machine cycles)
 DCR B ; Decrement B (1 machine cycle)
 JNZ MSEC-10-DELAY ; Jump if not zero (3 machine cycles)
 HLT ; Halt the processor
END

6. Differentiate between memory mapped I/O & peripheral mapped I/O or I/O mapped I/O. Draw the I/P or interfacing diagram of display 0 to 9 to the seven segment display.

=) Memory-Mapped I/O (MMIO):-

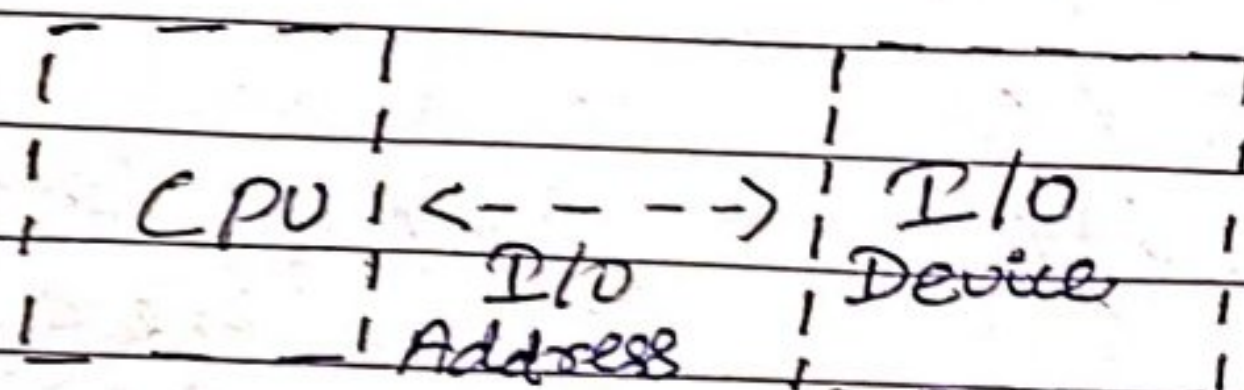
MMIO is a technique where I/O devices are treated as memory locations. These devices are assigned addresses in the memory address memory ^{space} locations. ^{accessing} These devices are assigned addresses allows you to read from or write to the corresponding devices. Memory mapped I/O simplifies the programming interface as the

Same instructions used for memory access can also be used for I/O operations

Peripheral-Mapped I/O

In peripheral-mapped I/O, separate instructions are used for I/O operations. Input & output devices are assigned unique addresses in the I/O address space. Dedicated instructions are used to transfer data between the CPU & the I/O devices differentiating I/O operations from memory operations.

Let's discuss interfacing a seven-segment display to demonstrate 0 to 9 using I/O mapped I/O



Assuming a hypothetical I/O address for controlling the seven-segment display, let's say:-

- I/O Address for Data: "0xF0"
- I/O Address for Control: "0xF1"

Code

```
MVP C, 09; Loop from 0 to 9.
MOV A, C; Move the current value to register A.
OUT 0xF0; O/P the data to the seven-segment display.
CALL DELAY; Call a delay subroutine.
MOV A, 0; Clear register A.
OUT 0xF1; Disable the display temporarily.
CALL DELAY; Call a delay subroutine.
DCR C; Decrement the loop counter.
JNZ START; Jump to the next iteration if not zero.
HLT; Halt of the processor.

DELAY; Delay Subroutine
MOV B, 0xFFH
DELAY-LOOP: DCR B
            JNZ DELAY-LOOP
            RET
```


7) Write a program to generate the 1st eight elements of the Fibonacci series. The Fibonacci series is defined as $F(i) = F(i-1) + F(i-2)$; $F(0) = 0$; $F(1) = 1$

=) ORG 0000H
 MOV B, 08; Set the loop counter to 8.
 MOV C, 00; Initialize $F(0)$ to 0.
 MOV D, 01; $F(1)$ to 1.

FIPO-LOOP;

MOV A, C; Load $F(i-1)$ into the accumulator.
 ADD D; Add $F(i-2)$ to the accumulator
 MOV E, A; Copy the result to register E ($F(i)$)

MOV A, D; Load $F(i-2)$ into the accumulator
 MOV D, E; Copy $F(i)$ to register D ($F(i-1)$)
 MOV C, A; Copy $F(i-2)$ to register C ($F(i-2)$)

; Output the Fibonacci no. in register E ($F(i)$).

; You can use out instruction or any other method based on your req.

DCRB; Decrement the loop counter.

JNZ FIPO-LOOP; Jump to the next iteration if not zero.

HLP; Halt the processor

END

8) Explain the instruction of 8085 that uses auxiliary carry flag.

=) In the 8085 processor, the auxiliary carry flag (AC) is used in arithmetic operations to indicate a carry out or borrow-out from the low-order nibble (4 bits) of the accumulator. The auxiliary carry flag is often utilized in BCD arithmetic.

~~Ques~~
One of the instructions that can affect the auxiliary carry flag is the DAA instruction. The DAA instruction is commonly used after addition operations involving BCD no. to adjust the result to a valid BCD representation.

DAA - Decimal Adjust Accumulator
= The DAA instruction

Q10) Draw the timing diagram for the instructions MOV A, M, IN RM, ADD B, LHL, IN, OUT, ORAM stored in memory location 9000H.

=)

Address	9000H	9001H	9002H	96

10) Explain the function of HOLD, HLDA, READY, RESET, SO, SI, ALE, CLK IN, CLK OUT.

=> HOLD (Hold) :-

function:- It is an input signal that indicates that another device is requesting control of the bus.

Usage:- The processor may relinquish control of the system bus when it receives a HOLD signal.

HLDA (Hold Acknowledge) :-

fun:- It is an output signal that acknowledges the HOLD request.

Usage:- Sent by the processor to indicate that it has granted control of the bus to the requesting device.

READY :-

fun:- An input signal indicating that external memory or I/O devices are ready to communicate with the processor.

The processor waits for the READY signal before completing its read or write operation.

RESET :-

initial state

fun:- It is an I/P signal used to reset the processor to its initial state. When the reset signal is activated, the processor initializes the internal registers & starts executing instructions from a predefined location.

SO, SI (Status Bits) :-

fun:- These are status bits that indicate the current state of the processor.

The combination of SO & SI bits defines the current operation or status of the processor, such as instruction fetch, memory read, memory write, etc.

⑧ ALE (Address Latch Enable) :-

Func:- An output signal used to latch the address from the multiplexed address/data bus onto the external address bus.

ALE is activated to indicate that the valid address is available on the bus.

⑨ CLK (Clock) :-

func:- It provides timing for the execution of instructions.

⑩ CLK IN, CLK OUT :-

CLK IN:- I/P signal for an external clock where it is used that provides the processor's

CLK out:- O/P signal for an external clock internal clock to other devices.

Usage:- CLK IN is used when an external clock is preferred over the internal clock. CLK OUT allows other devices to synchronize with the processor's clock.

11) Exp. the function of ADD, SBB, LHLD, XLCH, CMP, ADP, CMC, STA, DN instructions.

=> ADD:- Adds the content of the specified register or memory location to the accumulator.

SBB (Subtract Immediate with Borrow).

=> Subtracts an immediate value (with borrow) from the accumulator.

LHLD (Load HL Direct)

=> Loads 16-bit data from the specified memory address into the HL register pair.

LHLD addr loads the content of memory at the specified address into the HL register pair.

XCHG (Exchange HL with DE): -
 ⇒ Exchanges the content of the HL register pair with DE register pair. XCHG swaps the values in HL & DE registers.

CMP (Compare): -> Compares the content of the specified register or memory location with the accumulator.

ADD (Add Immediate): -> Adds an immediate value to the accumulator.

CMC (Complement Carry): -> Flips (complements) the carry flag.

STA (Store Accumulator): -> Stores the content of the accumulator into the specified memory address.

IN (Input): -> Reads data from an input port & loads it into accumulator.

12. What is interrupt? Explain diff. types of interrupts. What is RIM & SIM, & how it is executed? Write a program to enable all interrupt ~~except~~ except RST 5, 5.

⇒ Interrupt is a mechanism that allows a processor to temporarily halt the execution of its current program to handle a specific event or condition. Interrupts are crucial for handling real-time events & responding to external stimuli promptly.

Types of Interrupts: ->

1. Hardware Interrupts: -

External Interrupts: -> Triggered by external devices, such as a button press or sensor signal.

Internal Interrupts: -> Generated by internal events like timer overflow or divide-by-zero conditions.

2. Software Interrupts:-
~~Software~~ Invoked by executing a specific instruction.
On the program often used for system calls.

3. Maskable Interrupts:-
Can be disabled or enabled by the programmer
using the interrupt enable/disable flag.

4. Non-Maskable Interrupts:-
Cannot be disabled by the programmer.

Execution of RDM & SPM:-

1) RDM:- Program to enable ALL interrupt
except RST 5.5

```
ORG 8000H  
START: MVD B, 0FFH  
        OUT 7EH  
        HLT;
```

Interrupt service routine (ISR) for RST 5.5
RST 5.5 - ISR: JMP \$;
END START

* RDM (Read Interrupt Mask) & SPM (Set Interrupt Mask) instructions are specific to the 8085 ~~lp~~ processor architecture. They are used to read & set the status of various interrupt-related flags.

* Execution of RDM & SPM

- i) Both RDM & SPM instructions are executed by the ~~lp~~ in a single machine cycle.
- ii) They operate on the interrupt-related flags, providing a way for the programmer to control the interrupt system & retrieve status information.

9

13) What do you mean by maskable & non maskable interrupts of 8085? write a program to check whether RST 5.5 is pending. If it is pending, enable it without affecting the masking status of the other interrupts. write a program to clear all flag of 8085 l.p.

2) Maskable & Non-maskable :- Maskable Interrupts are those interrupts that can be individually enable or disabled by the programmer using specific instructions like SIM & RIM. These include INTK, RST 7.5 etc.

Non-maskable interrupts are those that cannot be disabled by programmer. The TRAP is an example of non-m in 8085 l.p.

* Program to check & Enable RST 5.5 if pending:

ORG 8000H

START;

RIM;

AND 00100000B; check the RST 5.5 interrupt
enable/disable flag.

JZ NO_RST5_5;

MVP A, 00000100B;

SIM;

NO - RST 5.5: HLT

'END ~~START~~.

* Program to clear All Flag of 8085 l.p.

ORG 8000H;

START;

SIM;

XRA A;

MVP B, 7;

CLR_LOOP: OUT 7EH;

DCR B;

JNZ CLR_LOOP

HLT

END;

- 1) To interface the 4kB EPROM, 2kB RAM & 4kB EPROM memory devices in 8085 μ p in absolute decoding method.
- 2) To interface a 4kB EPROM (27C32) a 2kB RAM, and another 4kB EPROM (27C32) with an 8085 μ p using absolute decoding you need to assign address ranges to each memory devices & control their chip select signals accordingly. Here's a basic ex-

ORG 8000H;

START; MOV A, 00H
OUT CS-EPROM1;
CALL READ-DATA;

MOV A, 01H;
OUT CS-RAM;
CALL WRITE-DATA;

MOV A, 02H;
OUT CS-EPROM2;
CALL READ-DATA

MOV A, 02H;
OUT CS-EPROM2;
CALL READ-DATA;
HLT;

READ-DATA: CALL RD-SIGNAL;
WRITE-DATA: CALL WR-SIGNAL;
RET

END;

15) CALL & RET are the only instructions of 8085 that uses subroutine. Comment on this statement. PUSH & POP are the only instructions of 8085 that uses stack. Comment on this statement. Write a program to clear the all flags without affecting the content of accumulator.

=) CALL & RET for Subroutines :-

=) The statement that "CALL & RET are the only instructions of 8085 that use subroutines" is not accurate. While CALL & RET are commonly used in implementing subroutines to create modular & reusable code, other instructions also contribute to the creation & execution of subroutines.

• PUSH & POP for Stack :-

=) The statement the "PUSH & POP are the only instructions of 8085 that use the stack" is ~~also~~ inaccurate. While PUSH & POP are primary instructions for managing the stack in the 8085 μp , other instructions such as CALL & RET, implicitly use the stack for storing the return address during subroutine calls. The stack is a crucial part of the subroutine calling mechanism, allowing the processor to remember the address to return to after the subroutine execution.

```
ORG 8000H ;
START ;
MVI A, 0FEH ;
PUSH PSW ;
XRA A ;
POP PSW ;
HLT ;
END ;
```


16) Explain the process of segmentation of memory in 8086 μ p. Adv. of Segmentation of memory. Draw & explain the flag register of 8086 μ p. Explain how to calculate 20 bit physical address. How Pipelining achieved in 8085 μ p. Explain the features of 8086 μ p.

=> Segmentation of 8086 μ p.

The 8086 μ p uses a memory segmentation model where the memory is divided into segments. Each segment is 64KB in size, & the processor uses a combination of a 16-bit segment address & 16-bit offset address to form a 20-bit physical address.

Segmentation Register:- The segment register holds the upper 16 bits of the effective address. Common segment register include CS (Code Segment), DS (Data Segment), SS (Stack Segment) & ES (Extra Segment).

Offset Register:- The offset register holds the lower 16 bits of the effective address. It is combined with the value in the segment register to generate the physical address.

Adv. of Segmentation Memory:-

- i) Memory Organization:- Segmentation allows efficient organization of memory enabling different segments for code, data & stack.
- ii) Addressability:- It provides a way to address more than 64KB of memory using a combination of segment & offset addresses.
- iii) Memory Protection:- Segmentation facilitates memory protection by segregating different types of data & code into distinct segments.

- preventing unintended accesses.
- iv) Dynamic Relocation:- The use of segments allows for dynamic relocation of programs in memory, making it easier to load & execute programs at diff addresses.
 - v) Modularity:- Segmentation supports modular programming by dividing code & data into separate segments, enhancing code reusability & maintenance.

Flag diagram $\rightarrow$ Q 4.

- * Calculation of 20-Bit physical Address.
 The 8086 forms a 20-bit physical address by combining the 16-bit segment address with the 16-bit offset address. The process is

$$\text{Physical Address} = \text{Segment} \times 16 + \text{Offset}.$$
 This combination results in a 20-bit address that can address up to 2^{20} or 1 MB of memory.

Pipelining in 8086 Microprocessor $\rightarrow$

- i) Execution Stage (EU):- Perform arithmetic & logic operations.
- ii) Instruction Prefetch & Decode stage (IPD):-
 Fetches & decodes the next instruction. The pipeline architecture allows the 8086 to overlap the execution of one instruction with the prefetching of the next instruction, improving overall instruction throughput.

Features of 8086 μ p.

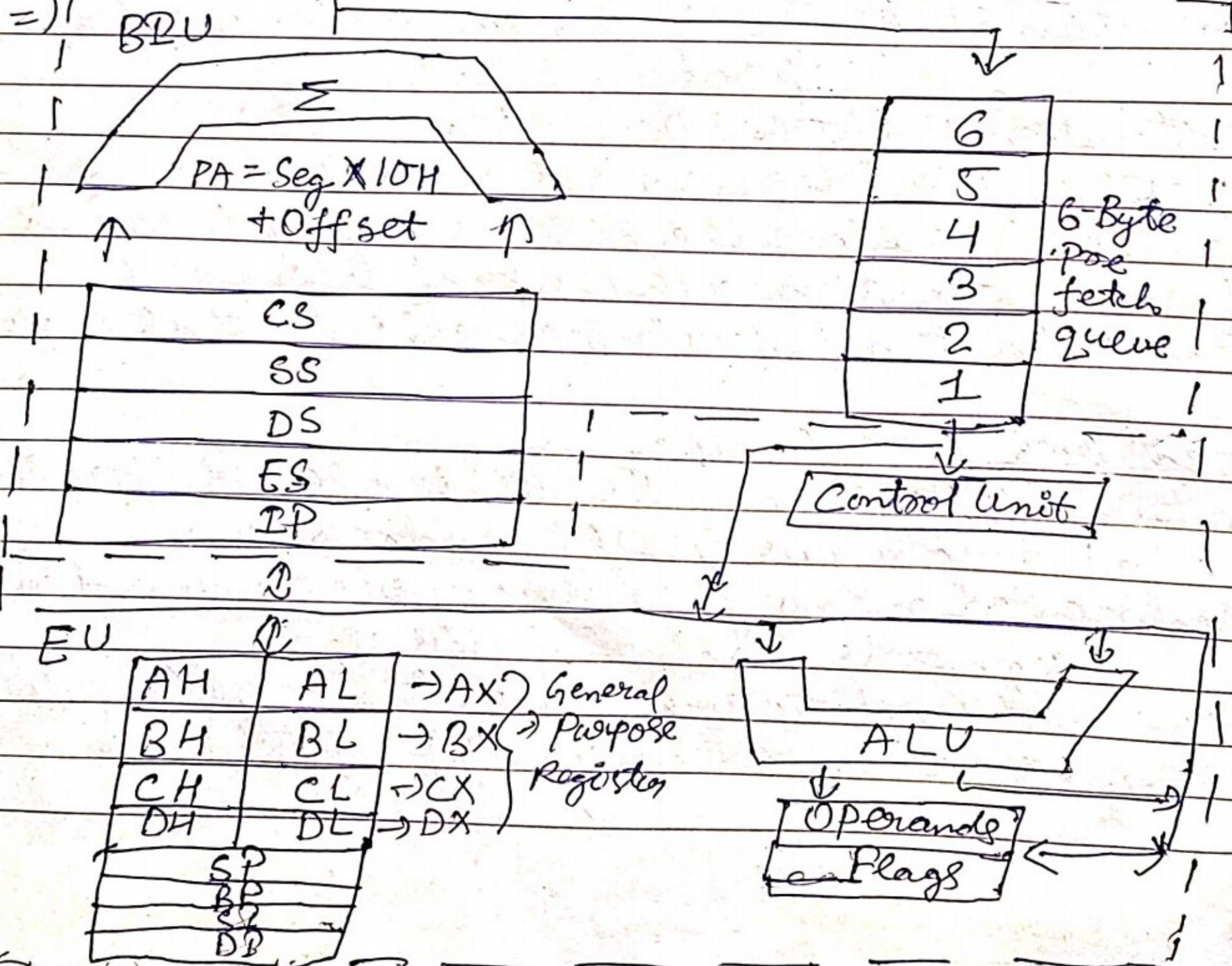
- i) 16-bit μ processor:- The 8086 is a 16-bit μ p with a 16-bit data bus & 20-bit address bus.
- ii) Segmented memory:- Memory is organized into segments, each of 64 KB size.
- iii) Pipelined Architecture:- Two stage pipeline for instruction execution & prefetching.

- iv) Registers: $\rightarrow$ It has a set of general purpose registers, segment registers & special purpose register like IP & flags.
- v) Instruction set: $\rightarrow$ A rich instruction set supporting various addressing modes, arithmetic, logical & control instructions.
- vi) Interrupts: $\rightarrow$ Supports hardware interrupts & has an interrupt vector table.
- vii) Address Bus Multiplexing: $\rightarrow$ The 20-bit physical address is generated by combining a 16-bit segment address with a 16-bit offset address.

17) Draw & discuss on brief the BPU & EU of 8086 μ P. What is data & address size of 8086? How 688 physical address is calculated? What is pipelining architecture of 8086 μ P? Explain the function of Queue register, AX, BX, CX & DX register.

Block diagram of 8086 μ P.

$\rightarrow$ To memory, Input/output



- 1) BUS Interface Unit (BIU) :-
- Responsible for handling external buses (address, data & control buses)
 - It includes the Segment Register, Instruction Pointer (IP) & Instruction Queue
 - The BIU fetches instructions & data from memory & loads them into the queue for execution.
 - It generates the physical address by combining the segment address & offset address.

- 2) Execution Unit (EU) :-
- Executes instructions fetched by the BIU.
 - Consists of the Arithmetic & Logic Unit (ALU) & various registers like AX, BX, CX, DX, SI, DI, SP & BP.
 - It performs arithmetic & logic operations on data.

* Data & Address Size in 8086 :-

- Data Size :- The 8086 is a 16-bit μ p, meaning it processes data in 16-bits chunks.
- Address Size :- The 8086 has a 20-bit address bus, allowing it to address 2^{20} or 1MB of memory.

Pipelining Architecture :-

- The 8086 μ p uses a dual-pipeline architecture with two stages: the BIU & EU.
- The BIU is responsible for instruction prefetching & fetching from memory, while the EU is responsible for executing instructions.
- This pipelining architecture allows the BIU to fetch the next instruction while the EU executes the current one, improving instruction throughput.

Registers in 8086 :-

1. Queue Register:

Holds the prefetched instruction bytes.

Allows the pipelining of instructions by prefetching the next instruction while the current one is being executed.

2) AX (Accumulation Register):-

- Primary register for arithmetic & logic operations.
- Often used for storing operands & results.

3) BX (Base Register):-

- Used as a base register for memory access.
- Often used in indexed addressing modes.

4) CX (Count Register):-

- Used in loop control & shift/rotate operations.
- Functions as a counter in string operations.

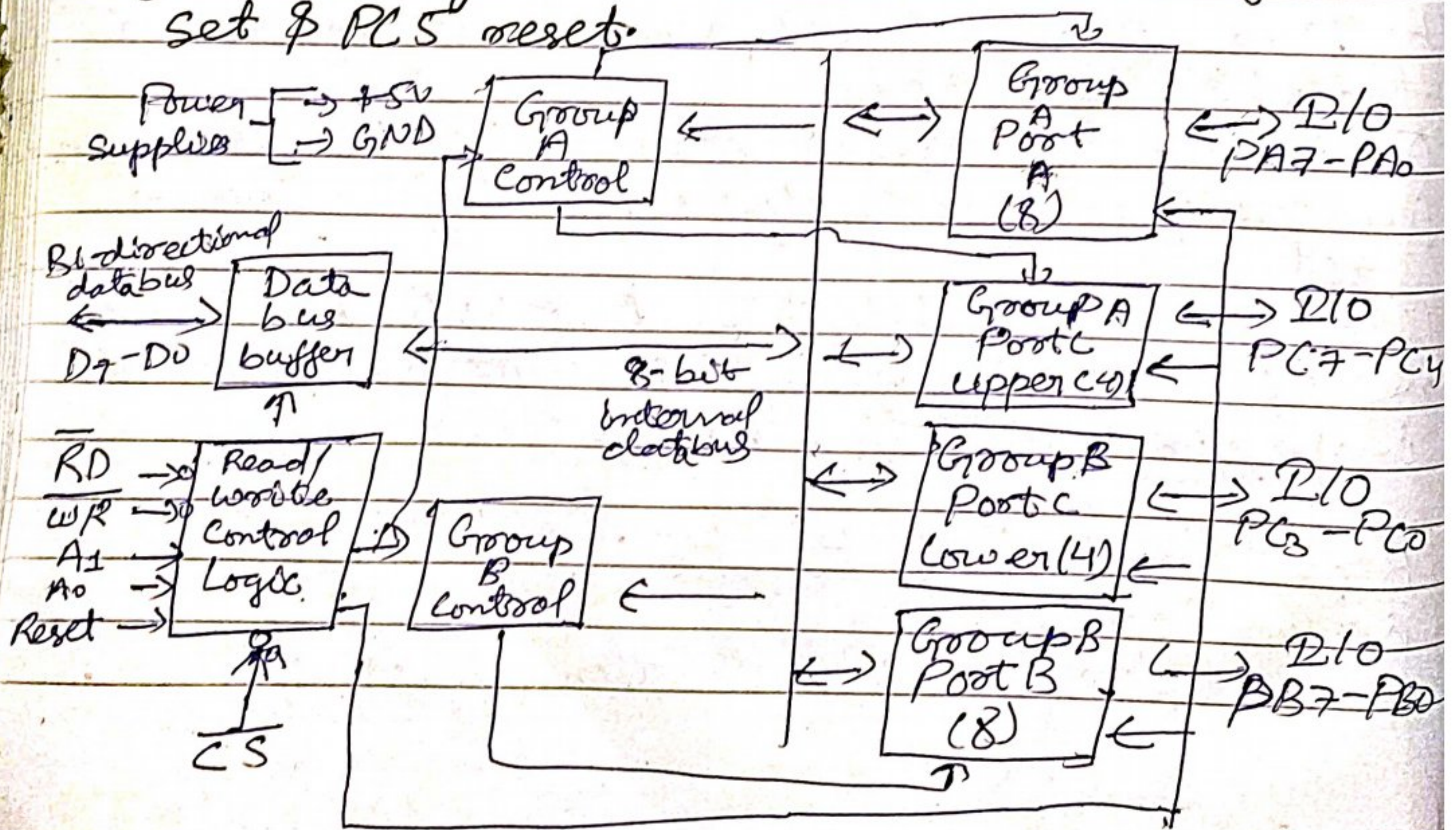
5) DX (Data Register):-

- Used in multiplication & division operations.
- Stores the most significant word of the results in multiplication & remainder in division.

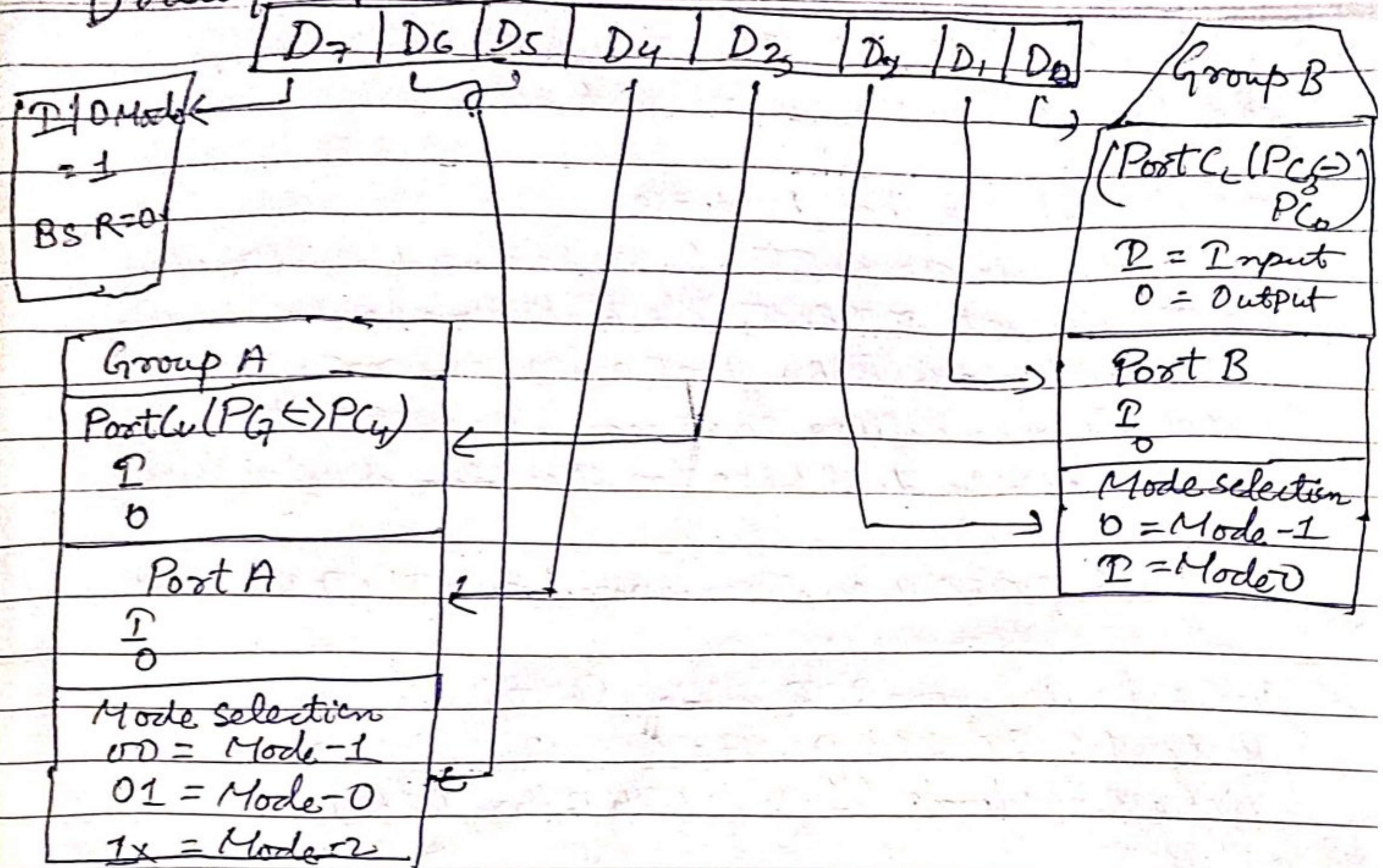
18) Draw & explain the functional block diagram of 8255.

Draw & write a control word for Port A & Port C as an input port & Port B as an output port.

Explain mode 1 operation of 8255 when used as an D/P & O/P port with proper diagrams. Explain the BSR mode of 8255. Write a control word for PC7 Set & PC5 reset.



Draw & explain the control word format of 8255



The 8255 is a programmable peripheral interface (PPI) chip that can be used to interface a μ processor with various I/P & O/p devices. It has 3-8-bit I/O ports (Port A, B, C).

* Control word for Port A:-

Byte 1: $SCS1=0$, $A=0$, $B=0$, $C=1$

Byte 2: Mode = 0010 (Mode 1 for Port A)

Control word for Port B:

Byte 1: $SCS1=0$, $A=0$, $B=1$, $C=0$

Byte 2: Mode = 0100 (Mode 2 for Port B)

Control word for Port C:

Byte 1: $SCS1=0$, $A=1$, $B=0$, $C=1$

Byte 2: Mode = 0010 (Mode 1 for Port C)

In Mode 1, Port A & Port C are configured as input ports & Port B is configured as output port.

BSR (Bit set/Reset) Mode

- In BSR mode, the 8255 allows individual bits of a port to be set or reset. The BSR mode can be configured using Mode 6 (BSR) for the desired port. The operation involves setting (BS) or resetting (BR) individual bits in the selected port.

Let's write a control word to set PC7 & reset PC5

Control word for BSR mode (Port C):

Byte 1: SC SI = 0, A = 1, B = 1, C = 0

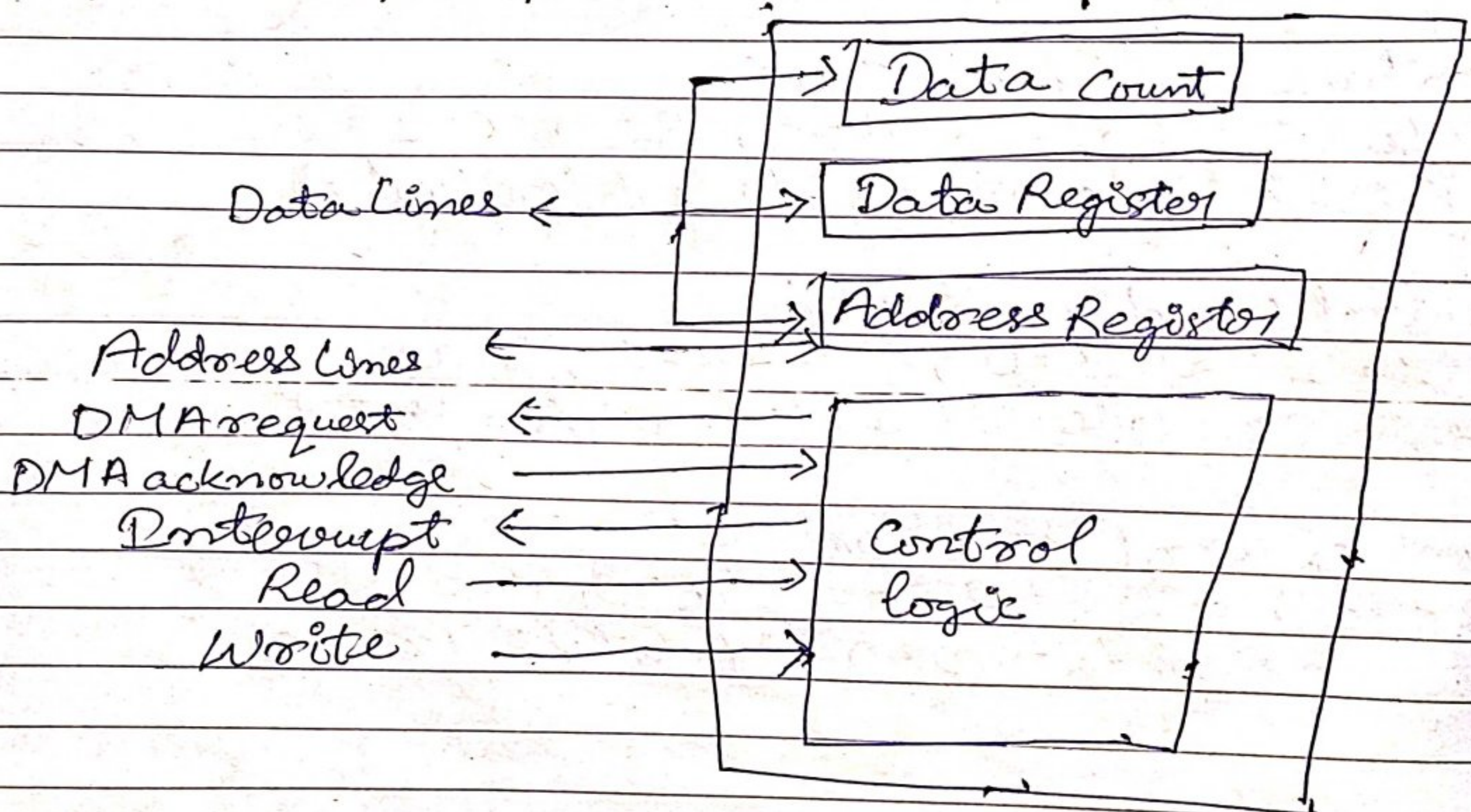
Byte 2: Mode 1100 (Mode 6 for BSR)

BSR Operation:

Set PC7: Write a logic 1 to PC7

Reset PC5: Write a logic 0 to PC5

19) Draw & Explain the DMA Operation.



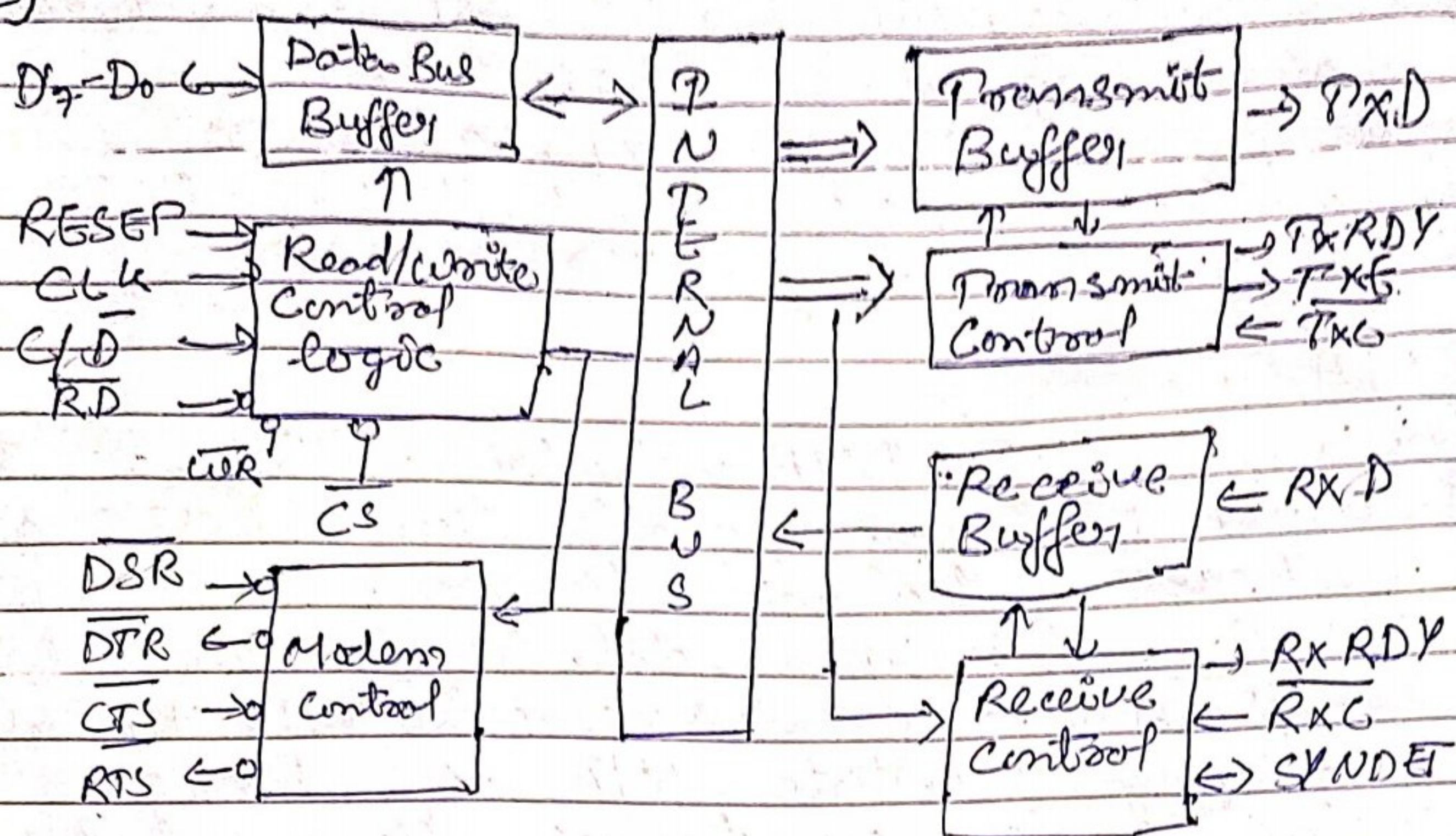
Direct Memory Access is a technique used in computer systems to allow peripherals to transfer data to & from the main memory without involving the CPU. DMA is particularly useful for high speed data transfers that can be handled more efficiently by dedicated DMA controllers rather than tying up the CPU.

DMA Operation:-

- i) Initialization:- The DMA controller is initialized by the CPU to specify the source & destination address, transfer size & other control information.
- ii) CPU Request:- When a data transfer is required, the peripheral or device sends a request to the DMA controller. The CPU may also initiate a DMA transfer based on certain conditions or through software control.
- iii) DMA Controller Request Acknowledgement
The DMA controller sends a request acknowledgement to the peripheral or device, indicating that it can proceed with the data transfer.
- iv) Bus Arbitration:- The DMA controller gains control of the system bus through a process called bus arbitration. This temporarily disconnects the CPU from the bus to allow the DMA controller to communicate directly with the memory & other peripherals.
- v) Data Transfer:- The DMA controller transfers data directly between the peripheral & the memory without involving the CPU.
The DMA controller manages the address lines, data lines & control signals to perform the data transfer.
- vi) Completion:- Once the data transfer is complete, the DMA controller signals the peripheral or device & releases control of the system bus.
The CPU may be notified of the completion status.

2705 2017 165
400

20) Draw & explain the 8251 USART & 8254 operations.



Architecture of 8251 operations

1. 8251 USART (Universal Synchronous/Asynchronous Receiver/Transmitter):

⇒ The 8251 is a popular programmable communication interface chip used for serial communication in μ p based systems. It can be configured to operate in either Synchronous or Asynchronous modes, making it versatile for various communication needs.

• Registers:-

Control Register (CR):- Used to configure the mode of operation.

Data Register (DR):- Holds the data to be transmitted or the received data.

• Operation:-

• Transmitting Data:-

1.1 Load the data into the Data Register (DR)

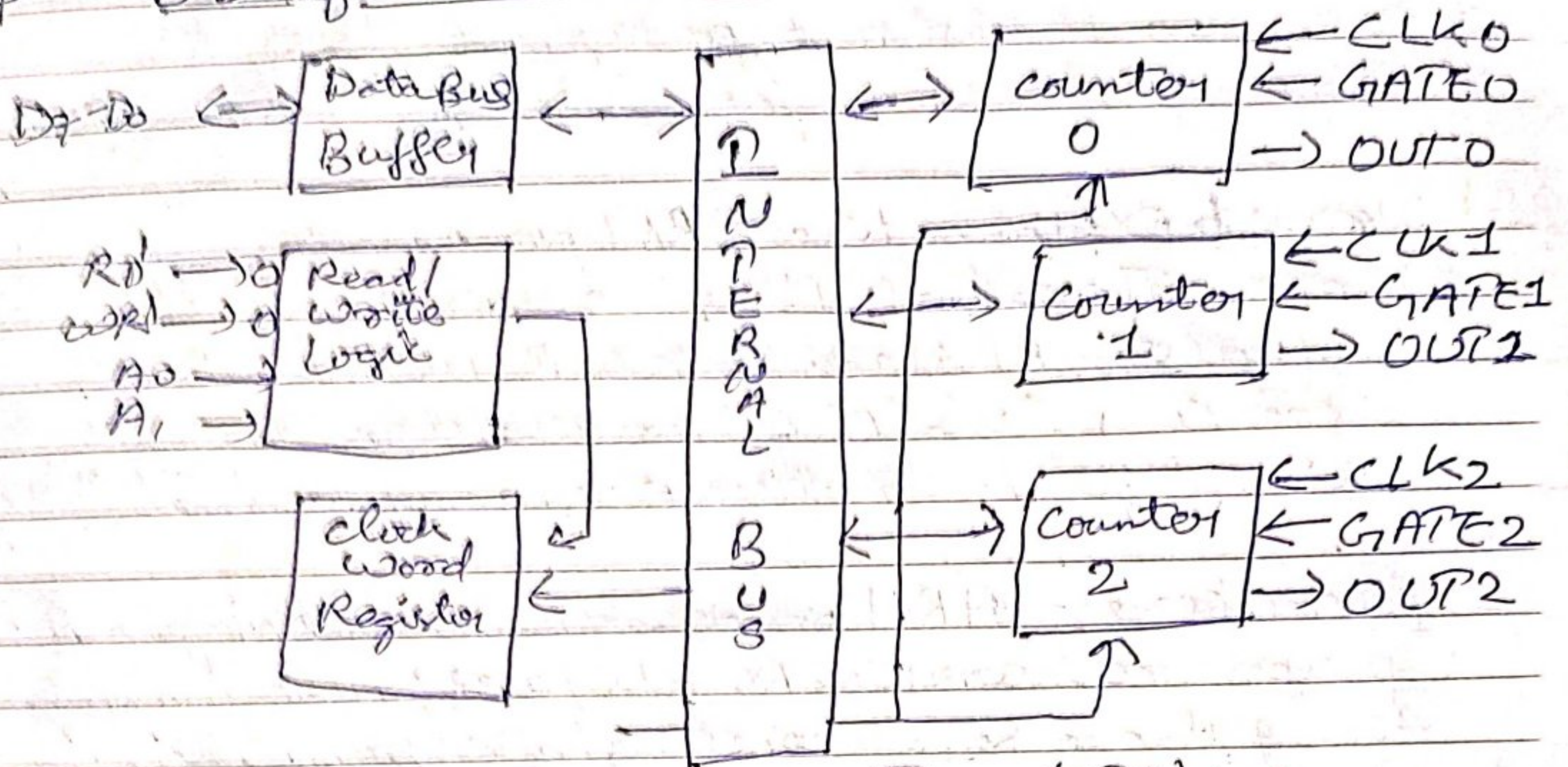
1.2 Set the appropriate control bits in the CR to configure the transmission parameters.

1.3 Trigger the transmission process.

• Receiving Data:-

- 1.1 set the control bits on the CR to configure the reception parameter.
- 1.2 wait for the receiver to indicate the readiness to receive.
- 1.3 Read the received data from the DR.

B.D of 8254 Timer



8254 programmable Interval Timer (PIT) :-

The 8254 is a programmable timer used for generating accurate time delays & generating periodic interrupts.

• Registers:-

• Counter 0, 1 & 2 Registers:- These registers hold the values of the three independent counters.

• Control Register:- Used to configure the mode of operation & to control the functioning of the counters.

• Operation:-

• Timer Mode:-

- 2.1. Load the initial count value into the count register.
- 2.2. Configure the control register to select the desired mode.
- 2.3. Start the timer.

22
2/1/21

Counting:-

- The timer will decrement its count value at a regular rate.
- When the count reaches zero an interrupt may be generated on the counter may automatically reload its initial value.

Applications :-

- Generating accurate time delays.
- Generating periodic interrupts for the O.P.
- Frequency division for clock generation

23) Draw & explain the ARM processor.

- ⇒ 1) Core:- The central CPU is the core of an ARM processor. ARM cores use a Reduced Instruction set Computing (RISC) architecture, which simplifies instruction execution & enhances performance.
- 2) Registers:- ARM processors have a set of general purpose registers (e.g. R0-R15) used for data manipulation & storage. Some registers have specific roles, such as the Program Counter (PC) & the stack pointer (SP).
- 3) Pipeline:- ARM processors typically employ a pipeline architecture, which allows the execution of multiple instructions simultaneously by breaking down the instruction execution into stages.
- 4) Instruction Set:- ARM processors have a rich & flexible instruction set, including arithmetic, logic & control flow instructions. This instruction set is designed to be efficient & compact.
- 5) Modes:- ARM processors have different operating modes such as User mode, Supervisor mode & IRQ mode. These modes allow the processor to execute instructions with different privilege levels.

6) Memory management Unit (MMU): - management
 → The MMU is responsible for virtual memory ~~memory~~ translation, converting virtual addresses to physical addresses & enable features like memory protection & demand paging.

7) Cache: - ARM processors often include cache memory to store frequently accessed data & instructions, reducing memory access latency & improving overall performance.

8) Interrupt Controller: → ARM processors have an interrupt controller that manages & prioritizes interrupts from various sources. This is crucial for handling events & ensuring efficient multitasking.

9) System Control Unit (SCU): → The SCU handles system-level control functions such as clock management, power management & system configuration.

10) Bus Interface: → It manages data transfer between the processor & other components such as memory & peripherals.

11) ARM Architecture Versions: → ARM processors come in various versions.

12) Thumb & Thumb-2: → ARM processors support the Thumb instruction set which uses 16-bit instructions to improve code density. Thumb-2 extends this to include a mix of 16-bit & 32-bit instructions for improved performance.

24) Draw the internal RAM structure of 8051 ~~MC~~ LLC. What is SFR & explain in details. Draw the pin out signals of 8051 LLC. Diff. between 8051 & 8051 LLC. Explain the features of 8051 LLC. Write a program in 8051 LLC to transfer a block of 10 data starting from 8000h of external ram location to internal ram location to internal ram location starting from 30h. Write a program in 8051 LLC to find ASCII value of a decimal no.

2) B Internal RAM structure of 8051 MC.
 The 8051 MC has 256 bytes of on-chip RAM, divided into two parts :- Register Banks & special function registers (SFRs). The Register Banks are used for general-purpose data storage, while the SFRs control various aspects of the MC's operation.

(SFR's) - They are a set of memory mapped registers that control the on-chip peripherals & other functionalities of the MC controller. They are accessed in the same way as RAM, but each register has a specific function. Ex. of SFR.

Pinout & Signal of 8051 MC.

- The 8051 MC typically has 40 pins, & the pinout includes power supply pins, I/O ports, oscillator pins, & other control pins. Some important signals include:
- VCC & GND :- Power supply pins.
 - P0 to P3 :- I/O ports
 - ALE (Address Latch Enable) :- Used to latch the lower address byte.
 - EA (External Access) :- Determines the source of code memory (internal or external).
 - XTAL1 & XTAL2 :- External crystal oscillator connections.

Program 1 :- Transfer of a block

RAM

```

ORG 8000H ; starting address of external
MOV DPTR, 8000H ; set DPTR to the source address
MOV R0, 30H ; set R0 to the destination address
MOV R1, 10 ; set the block size to transfer
TRANSFER_LOOP:
MOVX A, @DPTR ; Read data from external RAM
MOV @R0, A ; write data to internal RAM
INC DPTR ; Increment external RAM address
INC R0 ; Increment internal RAM address
DJNZ R1, TRANSFER_LOOP
END
  
```


Program to Find ASCII

MOV DPTR, #DECIMAL-NO.; SET DPTR to
the decimal NO

MOV A, #0; clear accumulator

DDU-LOOP;

< MOV B, #10; Set divisor to 10 in B

DDU AB; Divide A by B, quotient in A, remainder

ADD A, #30H; Convert remainder to ASCII

MOV @YOUR-DESTINATION, A; Store ASCII in
destination

INC YOUR DESTINATION, A; Move to the next
destination location

MOV A, B; Copy the quotient back to A

CJNE A, #0, DDU LOOP; Continue looping if quotient
is not zero

END;